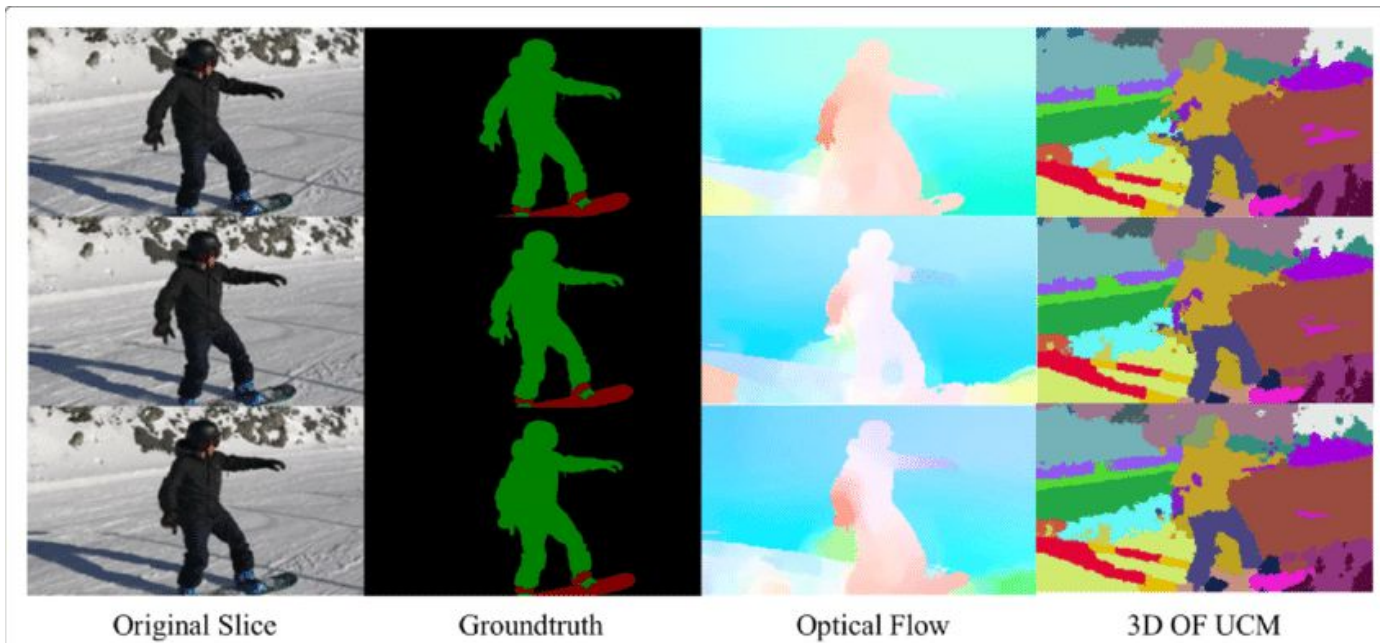


COMPUTER VISION

Motion Analysis



Motion Introduction

- Dealing with motion

- ▣ In real world

- Things actually move
 - Our perception system handles them

- ▣ In computer vision

- Sequence of static frames
 - With appropriate time gap, we see a nice motion

- ▣ We would like to give the machine, the same perception we have

- Machine to be able to understand the notion of motion

Motion Introduction

□ Video

▣ A sequence of frames captured relatively quickly (regular intervals) over time

■ Things do not change so much from one frame to the one immediately next

■ Each frame now becomes $I(x,y) \rightarrow I(x,y,t)$

■ Space and time

■ Space and time are different dimensions



Motion Field

- 2D representation of 3D motion
 - ▣ Each point is assigned a *velocity vector* corresponding to
 - Motion direction
 - Velocity
 - Distance from observer at an appropriate image location
 - ▣ Also called *velocity field*
 - ▣ Can also be acquired if the number of images in the sequence are small

Optical Flow

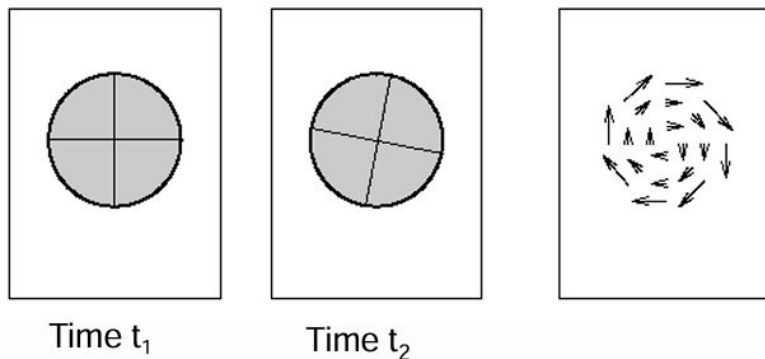
- A very small time distance between consecutive images needed
- No significant change occur in two consecutive frames
 - ▣ Main aim is to determine the motion field
 - Motion direction and motion velocity estimation for all image points
 - ▣ Optical flow might be different from true motion field
 - Illumination changes are reflected in optical flow

Motion Analysis

- Motion evaluation may or may not depend on object detection
 - ▣ Motion field – object dependent
 - ▣ Optical flow – object independent
- When image based information is available
 - ▣ Can be used for kernel-based object detection and tracking

Optical flow

- Reflects image changes due to motion during time interval dt
- *Optical flow field* – Motion fields/velocity field of the 3D motion of points across 2D image



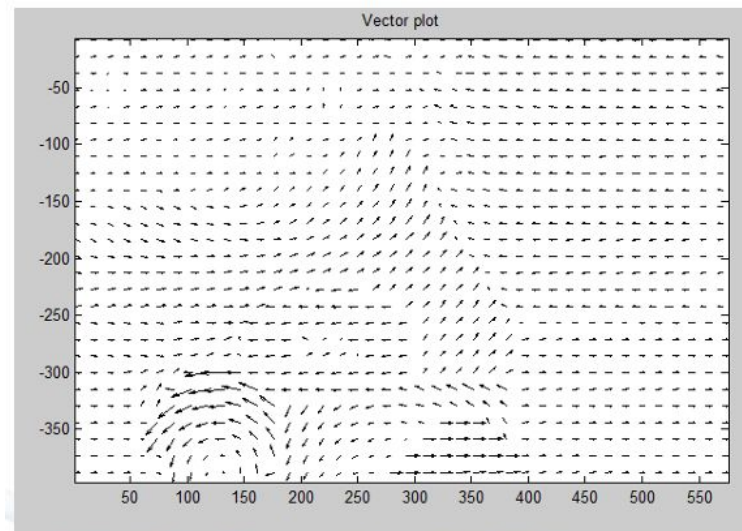
Optical flow

- Computation based on two assumptions
 - Observed brightness of any object point is constant over time
 - Nearby points in the image move in similar manner – *velocity smoothness*
- ▣ If these assumptions are violated, optical flow computation will produce errors

Optical flow



Optical flow



Optical flow



Image Source- Richard Szeliski

- Optical flow – Apparent motion
- Actual motion (underlying physical motion) – motion field

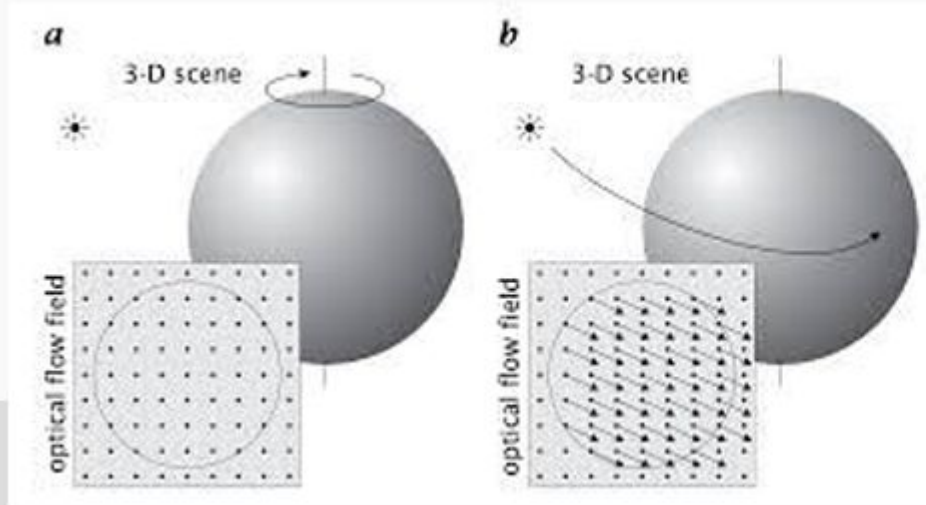
Global and local optical flow

- When brightness and velocity smoothness constraints are violated , errors will propagate
- Global error propagation is the biggest problem for global optical flow computation
- Local optical flow computation
 - ▣ Divide the image into small regions where the assumptions hold

Optical flow in motion analysis

- Optical flow can be used to study a large variety of motions
 - ▣ moving observer and static objects
 - ▣ static observer and moving objects
 - ▣ both moving
- Optical flow analysis does not result in motion trajectories

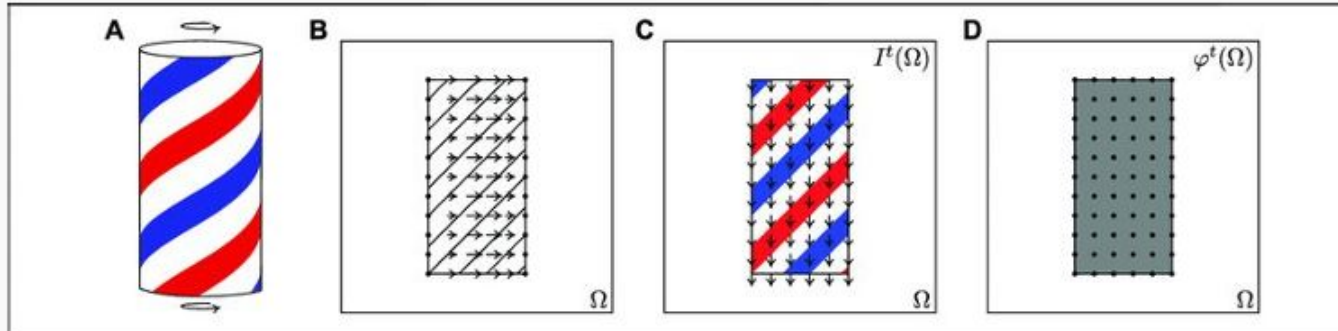
Optical flow in motion analysis (Is Optical flow = Motion everytime???)



a => motion field exists
No optical flow

b => no motion field exist
But optical flow exist

Optical flow in motion analysis



B - Motion field
D - Optical flow

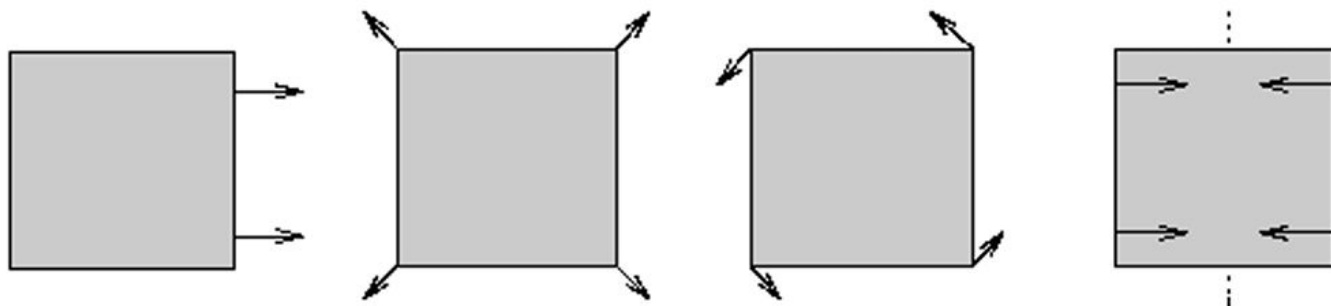
Optical flow in motion analysis

- Motion is usually combination of four basic elements
 - ▣ Translation at constant distance from the observer
 - ▣ Translation in depth relative to the observer
 - ▣ Rotation at constant distance about the view axis
 - ▣ Rotation of a planar object perpendicular to the view axis

Optical flow in motion analysis

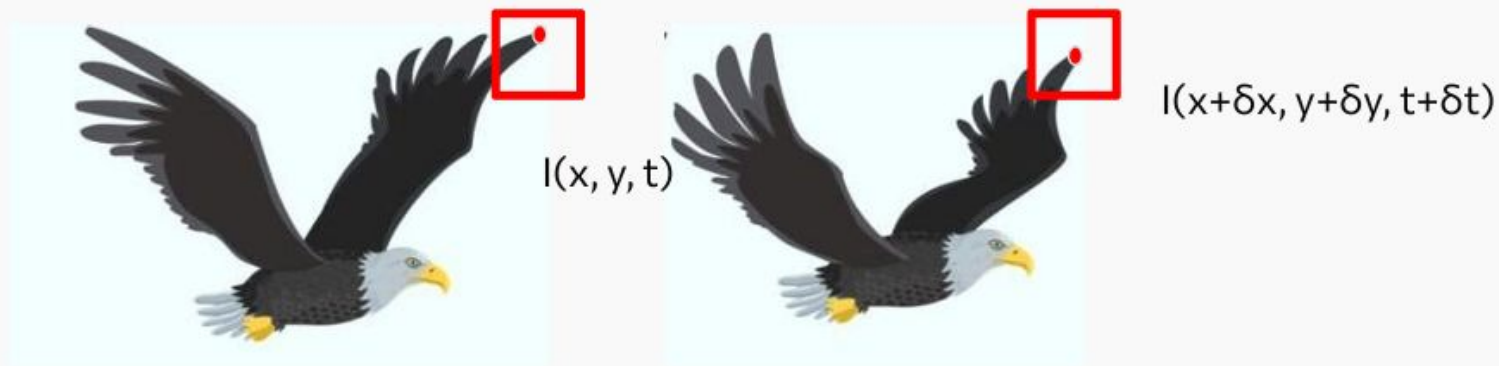
- ▣ Optical-flow based motion analysis is based on
 - Translation at constant distance is represented as a set of parallel motion vectors
 - Translation in depth forms a set of vectors having a common focus of expansion
 - Rotation at constant distance results in a set of concentric motion vectors
 - Rotation perpendicular to the view axis forms one or more sets of vectors starting from straight line segments

Optical flow in motion analysis



Translation at constant distance , Translation in depth relative to the observer
,Rotation at constant distance about the view axis , Rotation of a planar object
perpendicular to the view axis

Optical flow - Problem Definition



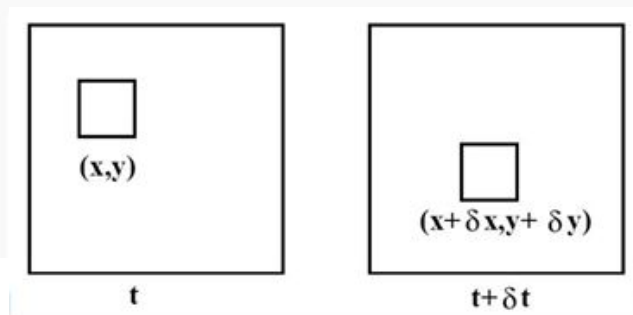
- We need to estimate pixel motion from image $I(x, y, t)$ to image $I(x+\delta x, y+\delta y, t+\delta t)$

Optical flow - Mathematical formalization

- Consider two images taken at time t and $t + \Delta t$
 - We express the **intensity** of each of its pixel as $I(x, y, t)$
- Assume $I(x, y, t)$ is the center pixel in a $n \times n$ neighbourhood and moves by δx , δy in time δt to $I(x + \delta x, y + \delta y, t + \delta t)$
 - Displacement = $(\delta x, \delta y)$

Optical flow $(u, v) = (\delta x / \delta t, \delta y / \delta t) \Rightarrow$ we have to find u and v

- From the assumptions
 - $I(x + \delta x, y + \delta y, t + \delta t) = I(x, y, t)$
Brightness assumption



Optical flow - Mathematical formalization

- Displacement δx , δy and time δt are small
 - $I(x+\delta x, y+\delta y, t+\delta t) \approx$ we have to take an approximation here for the intensity

We use “Taylor Series”

- $$I(x + \delta x, y + \delta y, t + \delta t) = I(x, y, t) + \frac{\partial I}{\partial x} \delta x + \frac{\partial I}{\partial y} \delta y + \frac{\partial I}{\partial t} \delta t + \dots$$

$$f(x + \delta x) = f(x) + \frac{\partial f}{\partial x} \delta x + \frac{\partial^2 f}{\partial x^2} \frac{\delta x^2}{2!} + \dots + \frac{\partial^n f}{\partial x^n} \frac{\delta x^n}{n!}$$

Optical flow

- Assuming the motion to be small the above equation can be developed in Taylor series as it follows:

$$I(x + \delta x, y + \delta y, t + \delta t) = I(x, y, t) + \frac{\partial I}{\partial x} \delta x + \frac{\partial I}{\partial y} \delta y + \frac{\partial I}{\partial t} \delta t + H.O.T.$$

- H. O. T. - Higher Order Terms
 - Assumed to be small and can safely be ignored.

Optical flow

- Brightness constancy assumption

$$I(x, y, t) = I(x + \delta x, y + \delta y, t + \delta t) \quad \text{=====> Equation 1}$$

- Taylor Series expansion (Small motion assumption)

$$I(x + \delta x, y + \delta y, t + \delta t) = I(x, y, t) + \frac{\partial I}{\partial x} \delta x + \frac{\partial I}{\partial y} \delta y + \frac{\partial I}{\partial t} \delta t \quad \text{=====> Equation 2}$$

- After substitution (1-2) -> optical flow constraint equation

$$-I_t = I_x \frac{\delta x}{\delta t} + I_y \frac{\delta y}{\delta t}$$

Optical flow

$$-I_t = I_x \frac{\delta x}{\delta t} + I_y \frac{\delta y}{\delta t}$$

- Direction of motion

$$c = \left(\frac{\delta x}{\delta t}, \frac{\delta y}{\delta t} \right) = (u, v)$$

Image gradient $\longrightarrow \nabla I \bullet c + I_t = 0$

- Brightness constancy constraint equation

$$I_x u + I_y v + I_t = 0$$

I_x , I_y and I_t can be computed or approximated

Spatial and Temporal derivatives

- Methods to compute derivatives needed in order to estimate optical flow
 - ▣ Need to compute the spatial derivatives of an image I at the time t
 - Sobel operator
 - ▣ The temporal derivative between time t and $t + \delta t$
 - Subtraction between frames at times t and $t + \delta t$

Optical flow methods

- Block-based methods
 - ▣ Minimizing sum of squared difference or sum of absolute difference, or
 - ▣ Maximizing normalized cross-correlation
- Differential methods of optical flow estimation
 - ▣ Based on partial spatial and temporal derivatives of the image signal

Optical flow methods

□ Differential methods

■ Lucas-Kanade method

- dividing image into patches and computing a single optical flow on each of them

■ Horn-Schunck method

- optimizing a functional based on residuals from the brightness constancy constraint, and a particular regularization term expressing the expected smoothness of the flow field

■ Buxton-Buxton method

- based on a model recovered from the motion of edges in image sequences

Optical flow methods

- Discrete optimization methods
 - ▣ The whole space is quantized, and every pixel is labelled, such that the corresponding deformation minimizes the distance between the source and the target image
 - ▣ The optimal solution is often computed through min-cut max-flow algorithms or linear programming

End of the Lecture.